SHUNYA OS

SYSTEM ALIGNMENT AUDIT

Architecture & Productization Layer Review

Generated: 2026-08-07 19:38 UTC

Commit: bc66b64 - BATCH-07-08

Executive Summary

This audit covers the SHUNYA OS productization layer — batches BATCH-05-06 (Intelligence + Abstraction) and BATCH-07-08 (Productization). The following architecture files were exported for review: the execution loop, decision engine, entity abstraction, communication processor, output generator, timeline service, entity API, and webhook ingestion routes.

File	Lines	Role
loop.py	189	Autonomous execution cycle — objects, commitments, leads, entities
decision_engine.py	239	Pure intelligence — returns next action for any object
entity.py	14	Generic Entity abstraction base model
processor.py	24	Inbound event to Message pipeline
generator.py	12	Structured output generation
timeline.py	33	Unified entity history view
entity_routes.py	29	REST API for entities + timeline
webhook_routes.py	19	External event ingestion endpoint

File: loop.py

189 lines, 6827 bytes

```
"""Continuous execution loop — observes objects and commitments, determines next action, executes, persist
s.
Emits pure observation signals: state_changed after update, no_action on noop.
Stateless, deterministic, no memory across cycles.
PROD-13: Relational execution graph propagation.
After a successful update, the loop checks for outbound relations and
propagates execution triggers to connected objects — one cycle, one chain.
No recursion, no business logic.
PROD-17: Autonomous execution loop — commitments.
The same cycle processes all commitments: gap-aware decision from the
latest observation, then applies it. Continuous processing, one cycle.
"""
import time
import logging
from app import db
from app.objects.models import Object
from app.runtime.decision_engine import get_next_action
from app.execution_engine.engine import execute_action
from app.signals.service import emit_signal
from app.graph.service import get_targets
from app.commitments.models import Commitment
from app.runtime.decision_engine import decide_next_from_commitment
from app.commitments.service import apply_decision
from app.runtime.decision_engine import decide_lead_task
from app.runtime.decision_engine import decide_lead_stage
from app.runtime.decision_engine import decide_entity
from app.communication.service import MessageService
from app.output.generator import generate_output
from app.communication.processor import process_inbound
from app.communication.delivery import deliver_messages
from app.core.entity import Entity
from app.models import Lead
logger = logging.getLogger(__name__)
_LOOP_INTERVAL = 3
def _propagate_to_targets(obj, summary: dict):
    """Propagate execution to all targets declared via object_relations.
    Pure structural propagation: every target gets one decision + execute
    cycle. Noop targets are skipped. Errors are recorded per-target.
    """
    relations = get_targets(obj.id)
    for rel in relations:
        try:
            target = Object.query.get(rel.target_object_id)
            if target is None:
                continue
            target_action = get_next_action(target)
            if target_action.get("type") == "noop":
                emit_signal(target.id, "no_action", {"state": target.state})
                summary["noops"] += 1
                summary["signals_emitted"] += 1
                continue
```

```
            state_before = dict(target.state or {})
            execute_action(target, target_action)
            emit_signal(
                target.id, "state_changed",
                {"from": state_before, "to": target.state},
            )
            summary["actions_taken"] += 1
            summary["signals_emitted"] += 1
        except Exception as e:
            logger.error(
                "Propagation error: source=%d target_id=%s: %s",
                obj.id, rel.target_object_id, e,
            )
            summary["errors"].append(
                {"object_id": rel.target_object_id, "error": str(e)},
            )
def run_cycle() -> dict:
    """Run one iteration of the execution loop.
    For each object:
    1. Determine next action via decision engine
    2. Execute action if update
    3. Propagate to relational targets
    4. Emit pure observation signal
    """
    objects = Object.query.all()
    summary = {
        "total_objects": len(objects),
        "actions_taken": 0,
        "noops": 0,
        "signals_emitted": 0,
        "errors": [],
    }
    for obj in objects:
        try:
            action = get_next_action(obj)
            if action["type"] == "noop":
                emit_signal(obj.id, "no_action", {"state": obj.state})
                summary["noops"] += 1
                summary["signals_emitted"] += 1
                continue
            state_before = dict(obj.state or {})
            execute_action(obj, action)
            emit_signal(
                obj.id, "state_changed",
                {"from": state_before, "to": obj.state},
            )
            summary["actions_taken"] += 1
            summary["signals_emitted"] += 1
            # PROD-52: after state update — generate output and create message
            output = generate_output(obj)
            MessageService.create(
                entity_id=obj.id,
                content=str(output),
```

```
                direction="outbound",
                channel="system"
            )
            # PROD-13: propagate to relational targets
            _propagate_to_targets(obj, summary)
        except Exception as e:
            logger.error("Loop error on object %d: %s", obj.id, e)
            summary["errors"].append({"object_id": obj.id, "error": str(e)})
    # PROD-17: process all commitments — gap-aware decision → apply
    commitments = Commitment.query.all()
    for c in commitments:
        decision = decide_next_from_commitment(c)
        apply_decision(c, decision)
    # PROD-28/30: process leads — task creation + outcome
    leads = Lead.query.all()
    for lead in leads:
        task_dec = decide_lead_task(lead)
        if task_dec.get("type") == "update":
            lead.outcome = "attempted"
        # PROD-32-34: stage progression
        stage_dec = decide_lead_stage(lead)
        # PROD-40: support multi-action list decisions
        if isinstance(stage_dec, list):
            for dec in stage_dec:
                if dec.get("type") == "update":
                    for k, v in dec.get("payload", {}).items():
                        setattr(lead, k, v)
        elif stage_dec.get("type") == "update":
            for k, v in stage_dec.get("payload", {}).items():
                setattr(lead, k, v)
    # PROD-59: process inbound events before entity processing
    process_inbound()
    # PROD-45: process all Entities — generic state transitions
    entities = Entity.query.all()
    for e in entities:
        ed = decide_entity(e)
        if ed.get("type") == "update":
            for k, v in ed.get("payload", {}).items():
                setattr(e, k, v)
    # PROD-59: deliver pending messages at end of cycle
    deliver_messages()
    db.session.commit()
    return summary
def run_loop(interval: int = _LOOP_INTERVAL, cycles: int = None):
    """Run the execution loop continuously."""
    count = 0
    while cycles is None or count < cycles:
        count += 1
        try:
            summary = run_cycle()
            if summary["actions_taken"] > 0 or summary["signals_emitted"] > 0:
                logger.info(
                    "Loop cycle %d: %d objects, %d actions, %d signals",
                    count, summary["total_objects"],
                    summary["actions_taken"], summary["signals_emitted"],
                )
        except Exception as e:
            logger.error("Loop cycle %d failed: %s", count, e)
        time.sleep(interval)
```

File: decision_engine.py

239 lines, 7216 bytes

```
"""Decision Engine — pure intelligence, returns next action for an object.
Supports multi-field payload evolution and structural multi-object intelligence.
No business assumptions — only structure, not meaning.
PROD-12: Single-link interaction (foundation).
An object may reference one other object via its `linked_to` state field.
When the referenced object reaches version 2, the linking object is synced.
PROD-13: Full object graph awareness.
An object may declare multiple structural relations:
    {
        "relations": [
            {"type": "depends_on", "target_id": 12},
            {"type": "blocks", "target_id": 15}
        ]
    }
Each relation is resolved when its target object reaches version 2.
Resolved relations accumulate in `resolved_relations` (non-destructive).
Once all declared relations are resolved, the object itself progresses to
version 2 so downstream objects can depend on it — graph propagation.
Relations are pure structure: {type, target_id}. No business meaning.
"""
from app.objects.models import Object
from app.commitments.models import Commitment
from app.observations.models import Observation
from app.models import Task
def build_context(entity):
    """Build a decision context dict for a lead/entity."""
    return {
        "state": getattr(entity, "stage", None),
        "outcome": getattr(entity, "outcome", None),
        "tasks": [],
        "observations": []
    }
def get_next_action(obj) -> dict:
    """Decide the next action for an object based purely on its state.
    Args:
        obj: Object instance with .state dict.
    Returns:
        update with payload if state progression needed.
        noop if state is fully evolved.
    """
    state = obj.state or {}
    if not state:
        return {
            "type": "update",
            "payload": {
                "initialized": True,
                "version": 1
            }
```

```
    }
# PROD-13 — structural multi-object graph awareness.
# Resolve every declared relation whose target has reached version 2.
# resolved_relations accumulates across cycles (non-destructive).
if state.get("relations"):
    resolved = set(state.get("resolved_relations") or [])
    newly = []
    for rel in state["relations"]:
        if not isinstance(rel, dict):
            continue
        target_id = rel.get("target_id")
        if target_id is None or target_id in resolved:
            continue
        target = Object.query.get(target_id)
        if target is not None and (target.state or {}).get("version") == 2:
            newly.append(target_id)
    if newly:
        accumulated = sorted(resolved | set(newly))
        return {
            "type": "update",
            "payload": {
                "resolved_relations": accumulated
            }
        }
    # All declared relations resolved → progress this object to version 2
    # so downstream objects can depend on it (graph propagation).
    if not state.get("initialized"):
        return {
            "type": "update",
            "payload": {
                "initialized": True,
                "version": 1
            }
        }
    if state.get("version") == 1:
        return {
            "type": "update",
            "payload": {
                "version": 2
            }
        }
# PROD-12 — structural single-link interaction (preserved).
# If this object references another object, and that object has reached
# version 2, bring this object into sync. No business logic.
if state.get("linked_to"):
    linked = Object.query.get(state["linked_to"])
    if linked is not None and (linked.state or {}).get("version") == 2:
        return {
            "type": "update",
            "payload": {
                "synced": True
            }
        }
if state.get("initialized") and state.get("version") == 1:
    return {
```

```
            "type": "update",
            "payload": {
                "version": 2
            }
        }
    return {"type": "noop"}
def decide_next_from_commitment(commitment: Commitment):
    """
    Decision based on latest observation.
    Returns:
        update_commitment with status=completed if latest observation
        is matched; noop otherwise.
    """
    # fetch latest observation
    obs = (
        Observation.query
        .filter_by(commitment_id=commitment.id)
        .order_by(Observation.id.desc())
        .first()
    )
    # no observation yet
    if not obs:
        return {"type": "noop"}
    # if matched → complete commitment
    if obs.status == "matched":
        return {
            "type": "update_commitment",
            "payload": {"status": "completed"}
        }
    # if deviated → mark failure
    if obs.status == "deviated":
        return {
            "type": "update_commitment",
            "payload": {"status": "failed"}
        }
    return {"type": "noop"}
def decide_lead_task(lead):
    """If a lead has no tasks, return a task creation action."""
    task_count = Task.query.filter_by(lead_id=lead.id).count()
    if task_count == 0:
        return {
            "type": "update",
            "payload": {"task": "Call customer"}
        }
    return {"type": "noop"}
def decide_lead_stage(lead):
    """Progress lead through lifecycle stages based on outcome.
```

```
    Uses build_context for unified context access.
    """
    context = build_context(lead)
    # PROD-39: multi-step decision for quoted leads
    if context["state"] == "quoted" and context["outcome"] != "closed":
        return [
            {"type": "update", "payload": {"task": "Follow up"}},
            {"type": "update", "payload": {"priority": "high"}}
        ]
    # RULE: contacted with no tasks → create "Send quote"
    if context["state"] == "contacted":
        task_count = Task.query.filter_by(lead_id=lead.id).count()
        if task_count == 0:
            return {
                "type": "update",
                "payload": {"task": "Send quote"}
            }
    if not context["outcome"]:
        return {"type": "noop"}
    if context["outcome"] == "attempted":
        if context["state"] == "new":
            return {
                "type": "update",
                "payload": {"stage": "contacted"}
            }
        if context["state"] == "contacted":
            return {
                "type": "update",
                "payload": {"stage": "quoted"}
            }
        if context["state"] == "quoted":
            return {
                "type": "update",
                "payload": {"stage": "closed"}
            }
    return {"type": "noop"}
def decide_entity(entity):
    """Generic decision for an Entity based on its state."""
    if entity.state == "new":
        return {"type": "update", "payload": {"state": "in_progress"}}
    return {"type": "noop"}
def explain_decision(entity, decision):
    return {
        "entity_id": entity.id,
        "decision": decision,
        "reason": "Derived from current state and context",
        "state": entity.state
    }
```

File: entity.py

14 lines, 402 bytes

```
"""Generic Entity base model — abstraction layer for any business entity."""
from app import db
class Entity(db.Model):
    __tablename__ = "entities"
    id = db.Column(db.Integer, primary_key=True)
    type = db.Column(db.String(50))
    state = db.Column(db.String(50))
    data = db.Column(db.JSON)
    def __repr__(self):
        return f"<Entity #{self.id} [{self.type}] state={self.state}>"
```

File: processor.py

24 lines, 648 bytes

```
from app.communication.inbound import InboundEvent
from app.communication.models import Message
from app import db
def process_inbound():
    events = InboundEvent.query.filter_by(processed=False).all()
    for e in events:
        # naive mapping (will evolve later)
        entity_id = e.payload.get("entity_id")
        if entity_id:
            msg = Message(
                entity_id=entity_id,
                content=str(e.payload),
                direction="inbound",
                channel=e.source,
                status="received"
            )
            db.session.add(msg)
        e.processed = True
    db.session.commit()
```

File: generator.py

12 lines, 310 bytes

```
def generate_output(entity):
    state = entity.state or {}
    if entity.type == "lead":
        return {
            "summary": f"Lead at stage: {state.get('stage')}",
            "next_action": state.get("next_step")
        }
    return {
        "summary": "Generic entity",
        "state": state
    }
```

File: timeline.py

33 lines, 918 bytes

```
from app.models import Task
from app.observations.models import Observation
from app.communication.models import Message
def get_entity_timeline(entity_id):
    events = []
    tasks = Task.query.filter_by(entity_id=entity_id).all()
    observations = Observation.query.filter_by(entity_id=entity_id).all()
    messages = Message.query.filter_by(entity_id=entity_id).all()
    for t in tasks:
        events.append({
            "type": "task",
            "data": t.description,
            "created_at": t.created_at
        })
    for o in observations:
        events.append({
            "type": "observation",
            "data": o.status,
            "created_at": o.created_at
        })
    for m in messages:
        events.append({
            "type": "message",
            "data": m.content,
            "created_at": m.created_at
        })
    return sorted(events, key=lambda x: x["created_at"])
```

File: entity_routes.py

29 lines, 792 bytes

```
from flask import Blueprint, jsonify
from app.core.entity import Entity
from app.core.timeline import get_entity_timeline
entity_bp = Blueprint("entity_api", __name__, url_prefix="/api/v1/entities")
@entity_bp.route("/", methods=["GET"])
def list_entities():
    entities = Entity.query.all()
    return jsonify([
        {"id": e.id, "type": e.type, "state": e.state}
        for e in entities
    ])
@entity_bp.route("/<int:entity_id>", methods=["GET"])
def get_entity(entity_id):
    e = Entity.query.get_or_404(entity_id)
    return jsonify({
        "id": e.id,
        "type": e.type,
        "state": e.state,
        "data": e.data
    })
@entity_bp.route("/<int:entity_id>/timeline", methods=["GET"])
def timeline(entity_id):
    return jsonify(get_entity_timeline(entity_id))
```

File: webhook_routes.py

19 lines, 476 bytes

```
from flask import Blueprint, request, jsonify
from app.communication.inbound import InboundEvent
from app import db
webhook_bp = Blueprint("webhook_api", __name__, url_prefix="/api/v1/webhook")
@webhook_bp.route("/", methods=["POST"])
def ingest():
    data = request.json or {}
    event = InboundEvent(
        source=data.get("source", "unknown"),
        payload=data
    )
    db.session.add(event)
    db.session.commit()
    return jsonify({"status": "received"})
```